

Week 5 Day 1 — Agent Foundations

The ReAct Loop

`run_agent` implements ReAct — **Reason** → **Act** → **Observe** → **repeat** — as a bounded while loop. Each iteration sends the conversation to the model and checks for `tool_calls`. None → return the final answer. Present → dispatch each through an `available_tools` registry, execute it, and append the result back as a role: "tool" message before looping again. Two safeguards make it production-viable: a `max_iterations` cap against runaway loops, and a `try/except` around the model call itself — which mattered in practice, since one test produced a malformed tool-call generation that would otherwise have crashed the process. Alongside conversation memory (messages), a separate **working memory** (state) tracks iteration count, tool calls, observations, and errors — for the application to debug, not for the model to see.

Tool Schemas Used

Two tools, each with a name, description, and JSON parameters schema:

- **calculator** — one expression string, evaluated via `ast.parse(..., mode="eval")` against a whitelist of operators (not `eval()`), safely rejecting both bad syntax and unsafe input like "import os".
- **get_weather** — one city string, looked up against a small dict; returns a structured `{"error": ...}` for unknown cities instead of raising.

Descriptions matter because they're the model's only signal for *when* to reach for a tool and *how* to fill its arguments — vague descriptions cause wrong tool selection or missed calls.

Failure Modes Observed

1. **Malformed function-call generation.** Asked "What's the weather like?" with no city, the model guessed "New York" but emitted a broken tool-call format, triggering a 400 error before our code even ran. *Fix:* `try/except` around the model call, degrade to a fallback message.
2. **Tool error handled honestly.** On a real error, the model retried once, then reported the failure honestly instead of fabricating data — better than expected, though the retry cost an iteration.
3. **Silent scope creep — most dangerous finding.** Asked to "send an email" with no such tool defined, the model didn't hallucinate a tool call — it silently did something else instead (fetched weather, wrote an email as text, never flagged the missing capability). `state["errors"]` stayed empty: no error signal at all. *Fix:* system prompt must state what the agent can't do.
4. **Malformed tool input,** tested directly, failed safely every time (bad syntax, `import os`, division by zero) — confirming the AST whitelist works.

5. **Non-convergence from a hard cutoff.** With `max_iterations=1`, the model gathered all needed data in one iteration but was cut off before reasoning over it — indistinguishable from a runaway loop by iteration count alone.

Why Frameworks Exist

Every guardrail above — safe dispatch, retries, loop limits, error handling — is a problem every agent project hits, and easy to get wrong under pressure. LangChain, LangGraph, and CrewAI don't do anything conceptually different from `run_agent()` here; they wrap the same loop in more tested scaffolding. Building it raw first is what makes that scaffolding legible instead of magic.